VOLUME 9 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 9

MULTITHREADING & CONCURRENCY

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 9

1. About This Module

Welcome to Volume 9 of the IT Study Hub Complete Java Programming Series. Up until now, every program you have written has executed sequentially. The computer processed Line 1, waited for it to finish, and then processed Line 2. However, modern computers have multiple CPU cores, and modern applications (like web servers, games, and streaming apps) must do thousands of things at the exact same time.

In this module, you will learn the art of **Multithreading and Concurrency**. You will learn how to split your application into multiple parallel workers (Threads) that execute code simultaneously, dramatically improving the speed and responsiveness of your software.

2. Learning Outcomes

- Understand the fundamental differences between Processes and Threads.
- Create and launch concurrent threads using the `Thread` class and `Runnable` interface.
- Manage the Thread Lifecycle using methods like `sleep()` and `join()`.
- Identify dangerous Concurrency Problems such as Race Conditions and Deadlocks.
- Protect shared data using `synchronized` blocks, `ReentrantLock`, and Atomic Variables.
- Establish Inter-Thread Communication using `wait()` and `notify()`.
- Upgrade from raw threads to professional Thread Pools using the `ExecutorService`.
- Retrieve asynchronous results using `Callable`, `Future`, and `CompletableFuture`.

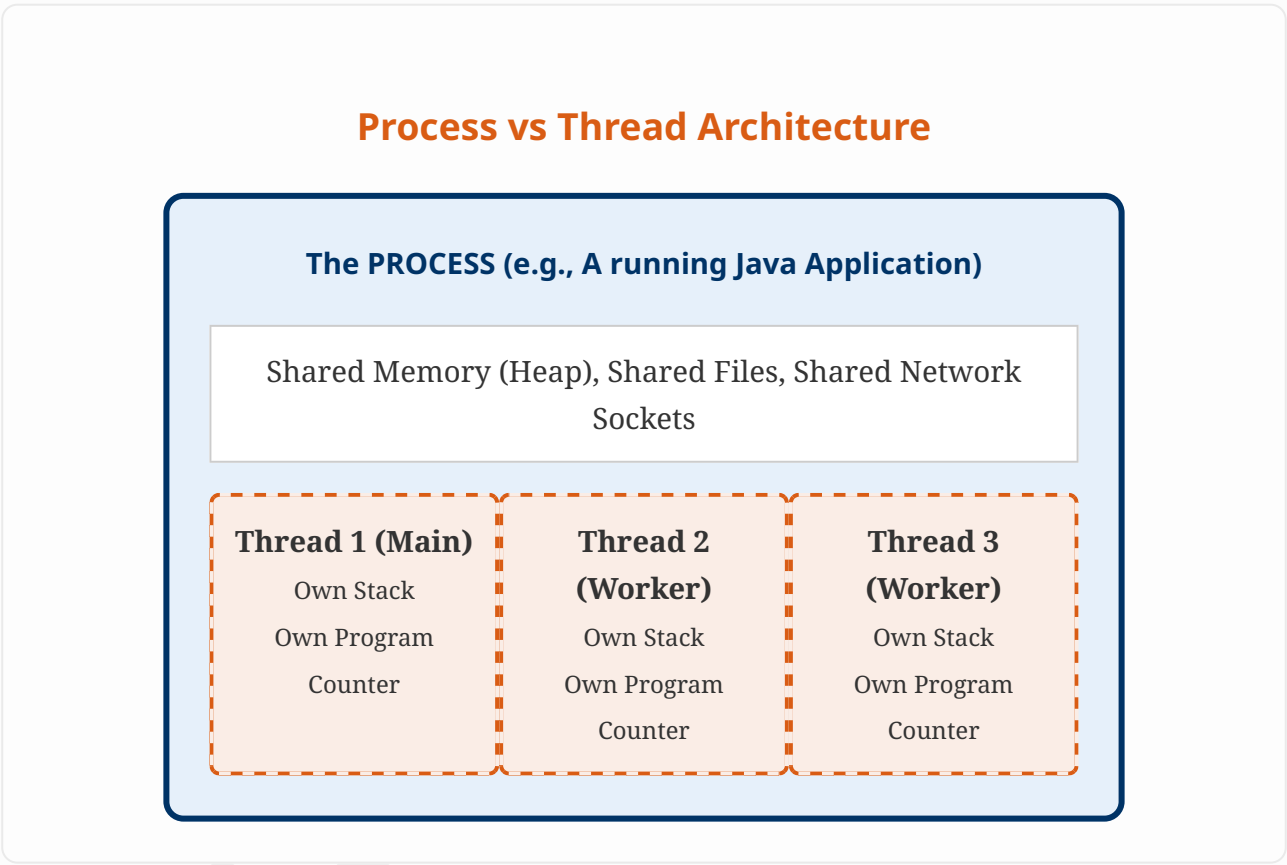
3. Module Coverage

We begin by creating raw threads to run tasks simultaneously. Next, we face the chaos of Race Conditions and learn how to tame them using Synchronization and Locks. We will then abandon manual thread creation for modern Java approaches, utilizing Thread Pools and ExecutorServices. Finally, you will build a "Multithreaded Task-Processing Application", handling asynchronous data seamlessly, followed by our extensive assessment section.

IT STUDY HUB

4. Processes vs Threads

To understand multithreading, we must first understand how an Operating System allocates resources to applications.



Process vs Thread Comparison

Feature	Process	Thread
Definition	An executing instance of a completely separate application.	A single path of execution *within* a process.
Memory	Heavyweight. Has its own separate memory space.	Lightweight. Shares memory (Heap) with other threads in the same process.
Communication	Inter-Process Communication (IPC) is slow and difficult.	Inter-Thread Communication is fast and easy (shared memory).
Crash Impact	If a process crashes, it usually doesn't affect other processes.	If a thread crashes unhandled, it can crash the entire process.

IT STUDY HUB

5. Creating Threads (Thread vs Runnable)

IT STUDY HUB

Concept: Extending the Thread Class

Creating a thread by inheriting from `java.lang.Thread`.

> ***Real-Life Analogy:***
> *Hiring a Chef who is strictly trained to only cook one specific recipe. They are a Chef (Thread) by nature, but they can't do anything else because they are locked into that identity.*

> **Simple Explanation:**
> You create a class that `extends Thread`. You must override the `run()` method with the code you want to execute simultaneously. To launch it, you instantiate the object and call `.start()`. NEVER call `.run()` directly!

Syntax / Structure:

```
class MyThread extends Thread {
    public void run() { /* concurrent code */ }
}
MyThread t = new MyThread();
t.start(); // Spawns the new thread!
```

Example Code:

```
class DownloadThread extends Thread {
    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName() + " is downloading a file...
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        System.out.println(Thread.currentThread().getName() + " has started.");

        DownloadThread t1 = new DownloadThread();
        t1.start(); // MAGIC HAPPENS HERE: A new thread is born!

        System.out.println(Thread.currentThread().getName() + " has finished.");
```

```
    }
}
```

```
Expected Output:
main has started.
main has finished.
Thread-0 is downloading a file...
```

Code Explanation:
Notice the output! The `main` thread launched `Thread-0` and instantly moved on to print its final message. `main` did NOT wait for `Thread-0` to finish. They ran in parallel. If we had called `t1.run()` instead of `t1.start()`, no new thread would be created, and `main` would have executed it synchronously.

Real-world Use:
Running a file download in the background while the user continues to interact with the app's user interface.

Common Mistakes:

- Calling `.run()` instead of `.start()`. This is a massive beginner mistake. `.run()` just executes the method normally on the main thread, defeating the whole purpose of multithreading.

Best Practices:

- Avoid extending `Thread`. Java only allows extending ONE class. If you extend Thread, your class cannot extend anything else. Use `Runnable` instead.

Practice Task:
Create a thread by extending `Thread` that prints numbers 1 to 5. Start it from main.

Concept: Implementing the Runnable Interface

Creating a thread by passing a 'Task' to a 'Worker'. (The Preferred Approach).

> ***Real-Life Analogy:***
> *Writing down a recipe on a piece of paper (Runnable task) and handing it to any available Chef (Thread) to cook. The recipe isn't a Chef, it's just a job.*

> **Simple Explanation:**
> You create a class that `implements Runnable` and override the `run()` method. Then, you create a new `Thread` object, hand your Runnable task to the Thread's constructor, and start the Thread.

Syntax / Structure:

```
class MyTask implements Runnable {
    public void run() { /* concurrent code */ }
}
Thread t = new Thread(new MyTask());
t.start();
```

Example Code:

```
class MusicPlayer implements Runnable {
    @Override
    public void run() {
        for(int i=1; i<=3; i++) {
            System.out.println("Playing Song " + i + "...");
        }
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        // 1. Create the Task
        MusicPlayer task = new MusicPlayer();

        // 2. Hire the Worker (Thread) and hand it the task
        Thread worker = new Thread(task, "Music-Thread");
```

```
        // 3. Start the worker
        worker.start();

        // Main thread does its own thing
        System.out.println("Main thread is browsing the internet.");
    }
}
```

```
Expected Output:
Main thread is browsing the internet.
Playing Song 1...
Playing Song 2...
Playing Song 3...
```

Code Explanation:
We separated the *Task* (`MusicPlayer`) from the *Runner* (`Thread`). This is much cleaner object-oriented design. The `main` thread prints its statement while the `Music-Thread` processes the loop concurrently.

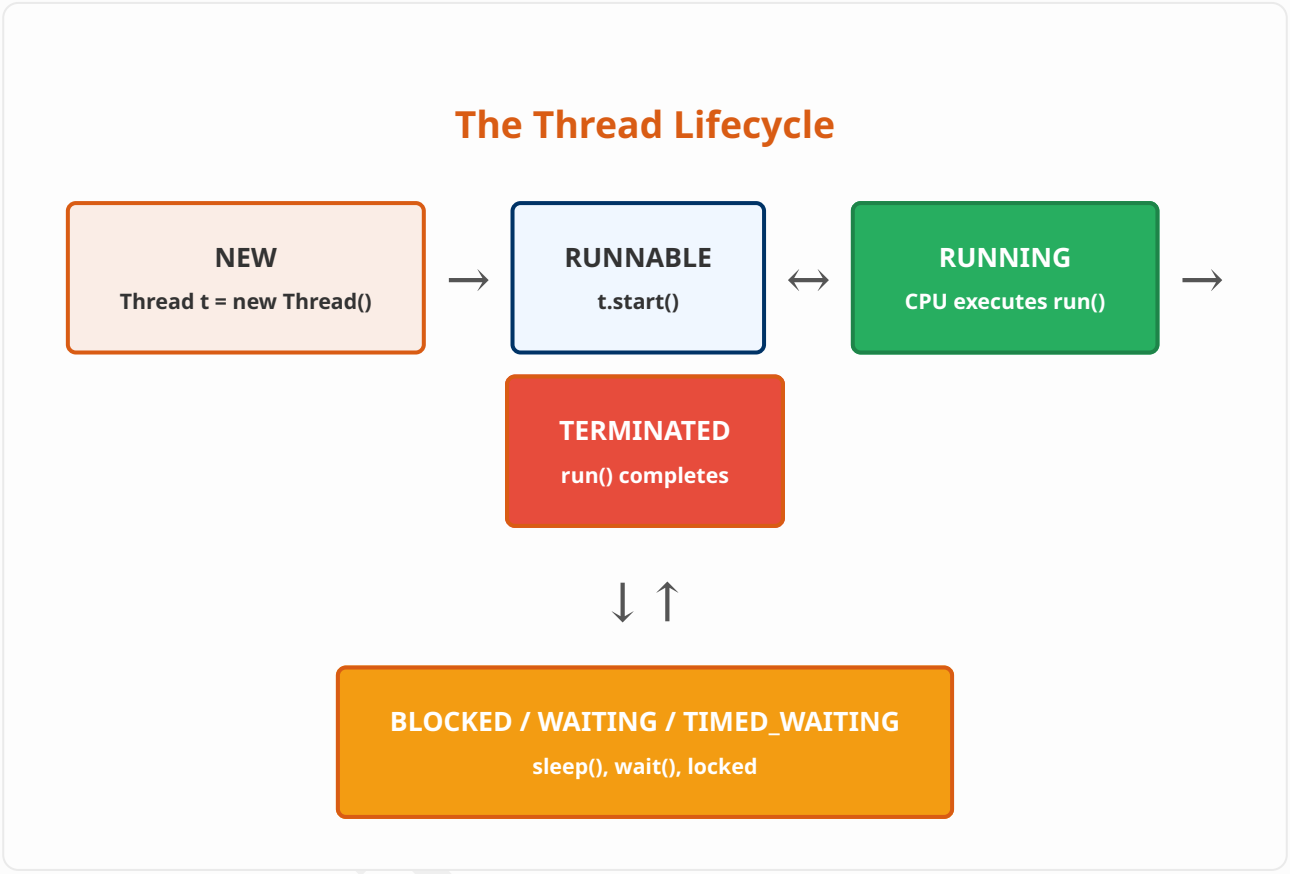
> ***Real-world Use:***
> *Passing different tasks (sending emails, processing payments) to a pool of background worker threads.*

> **Common Mistakes:**
> • Forgetting to wrap the Runnable inside a Thread object before trying to start it.

> **Best Practices:**
> • Always implement `Runnable` (or use a Lambda `Runnable`!) instead of extending `Thread`. It keeps your code loosely coupled.

Practice Task:
Write a Lambda expression that implements Runnable and prints 'Lambda Thread!'. Pass it to a Thread and start it.

6. Thread Lifecycle and Control Methods



Concept: Thread sleep() and join()

Controlling the pace of execution and forcing threads to wait for one another.

> ***Real-Life Analogy:***
> *`sleep()` is hitting the snooze button on an alarm clock for 5 seconds. `join()` is telling your boss, 'I cannot start my shift until my coworker finishes their shift.'*

> **Simple Explanation:**
> `Thread.sleep(milliseconds)` pauses the current thread for a specified time. `t.join()` forces the current thread to completely freeze and wait until thread `t` finishes its entire job.

Syntax / Structure:

```
Thread.sleep(1000); // Sleep for 1 second

Thread t1 = new Thread(task);
t1.start();
t1.join(); // The thread that calls this line waits for t1 to finish
```

Example Code:

```
public class SleepJoinDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            try {
                System.out.println("Worker: Processing heavy data...");
                Thread.sleep(2000); // Simulating 2 seconds of hard work
                System.out.println("Worker: Done!");
            } catch (InterruptedException e) {
                System.out.println("Worker was interrupted!");
            }
        });

        worker.start();

        System.out.println("Main: I need the worker's data. I will wait.");

        // Main thread FREEZES here until 'worker' finishes
```

```
            worker.join();

            System.out.println("Main: Worker is done, I can safely finish my job now.");
        }
    }
```

```
Expected Output:
Main: I need the worker's data. I will wait.
Worker: Processing heavy data...
(2 second pause)
Worker: Done!
Main: Worker is done, I can safely finish my job now.
```

Code Explanation:
Because we called `worker.join()`, the `main` thread paused execution. If we hadn't used `join()`, `main` would have printed its final line instantly, long before the worker finished its 2-second sleep.

Real-world Use:
`sleep()` is used in polling loops or retry mechanics. `join()` is used when aggregating data: Thread A calculates taxes, Thread B calculates discounts. Main Thread MUST `join()` them both before printing the final receipt.

Common Mistakes:

- Forgetting that `sleep()` and `join()` throw a checked `InterruptedException`. You MUST wrap them in a `try-catch` block or declare `throws`.

Best Practices:

- Avoid relying on arbitrary `sleep()` timings to sync threads. A computer's speed fluctuates. Use `join()` or proper synchronization tools instead.

---

7. The Chaos of Concurrency (Race Conditions)

IT STUDY HUB

Concept: Race Conditions and Shared Data

When multiple threads access and modify the same variable simultaneously, resulting in corrupted, unpredictable data.

> ***Real-Life Analogy:***
> *Two people have debit cards connected to the exact same bank account. The account has $100. Both people check the balance at the exact same millisecond. Both see $100. Both withdraw $100 simultaneously. The bank ends up giving away $200 because the two actions 'raced' and overlapped, bypassing the math logic.*

> **Simple Explanation:**
> Operations like `count++` actually involve three steps: 1. Read the current value. 2. Add 1. 3. Write the new value. If Thread A reads '5', and before it writes '6', Thread B also reads '5', they will both write '6'. A count was lost!

Example Code:

```
class Counter {
    int count = 0;

    // Non-synchronized method
    public void increment() {
        count++;
    }
}

public class RaceConditionDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();

        // Create a task that increments 1000 times
        Runnable task = () -> {
            for(int i=0; i<1000; i++) c.increment();
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
```

```
        t2.start();

        t1.join();
        t2.join();

        // We expect 2000, but we will get a random, lower number!
        System.out.println("Final Count: " + c.count);
    }
}
```

```
Expected Output:
Final Count: 1845
```

Code Explanation:
The output is NOT 2000. It might be 1845, or 1920, or 1200. Because `t1` and `t2` execute `count++` at the exact same microsecond, they overwrite each other's work. This is a fatal logical bug.

> ***Real-world Use:***
> *Ticket booking systems. If two users click 'Book Last Seat' at the exact same millisecond, an unprotected system will double-book the seat.*

> **Common Mistakes:**
>
> - Assuming `count++` is a single atomic operation. It is not. It can be interrupted halfway through by the CPU.

> **Best Practices:**
>
> - Never let multiple threads modify shared, unprotected variables. Use Synchronization.

---

8. Synchronization & Locks

IT STUDY HUB

Concept: The 'synchronized' Keyword

A keyword that places a lock on a method or block of code, ensuring only ONE thread can execute it at a time.

> ***Real-Life Analogy:***
> *A public restroom with one toilet. `synchronized` is the lock on the door. If Thread A goes in and locks the door, Thread B must wait outside in a queue until Thread A unlocks the door and leaves.*

> **Simple Explanation:**
> By adding the word `synchronized` to a method, Java forces threads to take turns. Thread A acquires the 'Intrinsic Lock' of the object. Thread B is blocked. When Thread A finishes, it releases the lock, and Thread B grabs it.

Syntax / Structure:

```
public synchronized void increment() {
    count++;
}
```

Example Code:

```
class SafeCounter {
    int count = 0;

    // SYNCHRONIZED method locks the object
    public synchronized void increment() {
        count++;
    }
}

public class SyncDemo {
    public static void main(String[] args) throws InterruptedException {
        SafeCounter c = new SafeCounter();

        Runnable task = () -> {
            for(int i=0; i<1000; i++) c.increment();
        };
```

```
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start(); t2.start();
        t1.join(); t2.join();

        // Will be EXACTLY 2000 every single time!
        System.out.println("Final Safe Count: " + c.count);
    }
}
```

```
Expected Output:
Final Safe Count: 2000
```

Code Explanation:
Because `increment()` is synchronized, `t1` and `t2` cannot enter the method at the same time. `t1` enters, locks it, does `count++`, and leaves. Then `t2` enters. The Race Condition is completely cured.

> ***Real-world Use:***
> *Protecting the code block that deducts money from a user's bank account to prevent double-spending.*

> **Common Mistakes:**
>
> - Synchronizing everything. Synchronization destroys parallel performance because it forces threads into a single-file line. Only synchronize the exact lines of code that touch shared variables.

> **Best Practices:**
>
> - Use `synchronized` blocks instead of methods when possible to minimize the locked area: `synchronized(this) { count++; }`.

Concept: Explicit Locks (ReentrantLock)

An advanced, flexible alternative to the `synchronized` keyword provided by the `java.util.concurrent.locks` package.

> ***Real-Life Analogy:***
> *`synchronized` is an automatic door lock that locks when you enter and unlocks when you leave. `ReentrantLock` is a manual padlock. You manually call `.lock()` and MUST manually call `.unlock()`. It's more dangerous, but allows you to lock the door from the outside, pass the key around, or check if the door is currently locked before trying to enter.*

> **Simple Explanation:**
> Explicit locks give you features that `synchronized` cannot: you can attempt to lock but give up after 5 seconds (`tryLock()`), or separate read/write access.

Syntax / Structure:

```
Lock lock = new ReentrantLock();
lock.lock();
try {
    // critical section
} finally {
    lock.unlock(); // MUST be in finally block!
}
```

Example Code:

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class LockedCounter {
    private int count = 0;
    private Lock lock = new ReentrantLock(); // Create manual lock

    public void increment() {
        lock.lock(); // Manually acquire lock
        try {
            count++;
        } finally {
```

```
            lock.unlock(); // GUARANTEE the lock is released
        }
    }
    public int getCount() { return count; }
}
```

```
Expected Output:
[Output identical to synchronized: 2000]
```

Code Explanation:
We manually acquire the lock before touching `count`. The `finally` block is absolutely critical. If `count++` magically threw an exception and crashed, the `finally` block ensures `lock.unlock()` still runs. Without it, the lock remains locked forever, and all other threads freeze permanently (Deadlock).

> ***Real-world Use:***
> *High-performance gaming engines where you need fairness policies (guaranteeing the longest-waiting thread gets the lock next).*

> **Common Mistakes:**
> • Forgetting to put `lock.unlock()` inside a `finally` block.

> **Best Practices:**
> • Stick to `synchronized` for simple tasks. Use `ReentrantLock` only when you need advanced features like `tryLock()` or Fairness.

---

9. Inter-Thread Communication (wait & notify)

IT STUDY HUB

Concept: wait() and notify()

Methods used by threads to pause and signal each other, allowing them to coordinate complex tasks.

> ***Real-Life Analogy:***
> *A Chef (Producer) and a Waiter (Consumer). The Waiter goes to the kitchen. There is no food. The Waiter calls `wait()` (goes to sleep). The Chef cooks the food, places it on the counter, and calls `notify()` (rings the bell). The Waiter wakes up and serves the food.*

> **Simple Explanation:**
> These methods MUST be called inside a `synchronized` block. `wait()` causes the current thread to drop the lock and go to sleep. `notify()` wakes up one sleeping thread and tells it to try grabbing the lock again.

Syntax / Structure:

```
synchronized(obj) {
    while(condition) obj.wait();
    // do work
    obj.notifyAll();
}
```

Example Code:

```
class MessageDrop {
    private String message;
    private boolean hasMessage = false;

    public synchronized void put(String msg) throws InterruptedException {
        while (hasMessage) {
            wait(); // Wait until the previous message is read
        }
        message = msg;
        hasMessage = true;
        notify(); // Wake up the reader!
    }

    public synchronized void take() throws InterruptedException {
        while (!hasMessage) {
```

```
            wait(); // Wait until a message arrives
        }
        System.out.println("Read message: " + message);
        hasMessage = false;
        notify(); // Wake up the writer!
    }
}
```

```
Expected Output:
[Threads will perfectly alternate: Put -> Take -> Put -> Take]
```

Code Explanation:
This is the classic Producer-Consumer problem. If the Reader arrives first, `hasMessage` is false, so it calls `wait()` and freezes. When the Writer writes a message, it sets `hasMessage` to true and calls `notify()`. The Reader wakes up, escapes the while-loop, prints the message, and calls `notify()` to wake up the Writer for the next round.

Real-world Use:
Data processing pipelines. Thread A downloads chunks of video. Thread B processes the video. Thread B must `wait()` if Thread A hasn't downloaded the next chunk yet.

Common Mistakes:

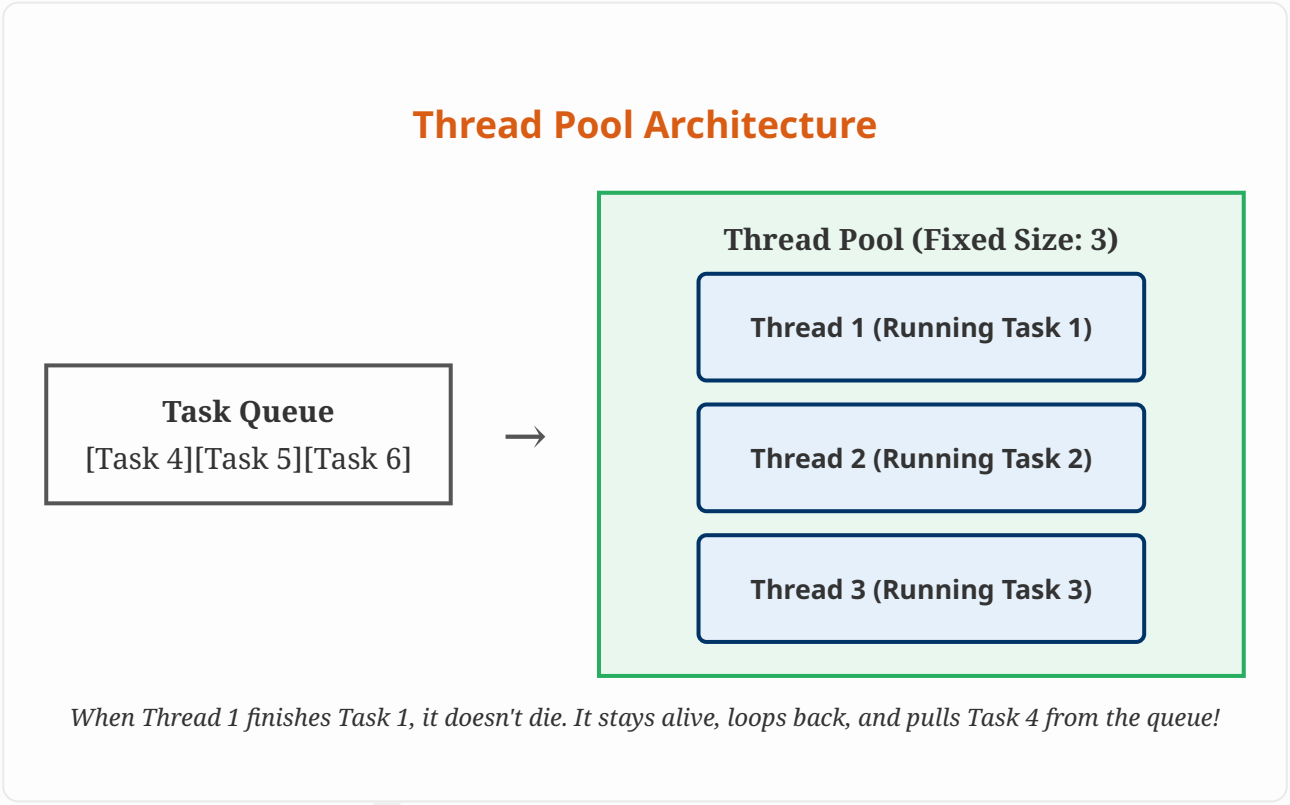
- Calling `wait()` or `notify()` outside of a `synchronized` block. This throws an `IllegalMonitorStateException`.
- Using an `if` statement instead of a `while` loop to check the wait condition. (Threads can suffer from 'Spurious Wakeups', so they must re-check the condition in a while loop after waking up).

Best Practices:

- Modern Java almost never uses manual `wait()` and `notify()`. Instead, we use `BlockingQueue` from the `java.util.concurrent` package, which handles all this complex signaling automatically safely.

10. The Executor Framework (Thread Pools)

Creating a new `Thread` manually using `new Thread(task).start()` is highly inefficient. Threads require OS resources to boot up. If a web server receives 10,000 requests and creates 10,000 threads, the server's RAM will explode, and the system will crash.



When Thread 1 finishes Task 1, it doesn't die. It stays alive, loops back, and pulls Task 4 from the queue!

Concept: ExecutorService

A high-level framework that manages a pool of reusable worker threads.

> ***Real-Life Analogy:***
> *A Call Center. You don't hire a new employee every time the phone rings and fire them when they hang up. You hire 10 fixed operators. If 15 calls come in, 10 get answered, and 5 wait in a queue until an operator is free.*

> **Simple Explanation:**
> You ask `Executors` to create a pool of threads. You then `.submit()` your Runnable tasks to the pool. The ExecutorService handles the distribution of tasks, the execution, and keeps the threads alive for future use.

Syntax / Structure:

```
ExecutorService executor = Executors.newFixedThreadPool(3);
executor.submit(new MyTask());
executor.shutdown();
```

Example Code:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolDemo {
    public static void main(String[] args) {
        // Create a pool of exactly 2 reusable threads
        ExecutorService pool = Executors.newFixedThreadPool(2);

        // Submit 5 tasks to the pool
        for (int i = 1; i <= 5; i++) {
            final int taskId = i;
            pool.submit(() -> {
                System.out.println("Executing Task " + taskId + " via " + Thread.current
            });
        }

        // Prevent new tasks from being submitted and shutdown gracefully
        pool.shutdown();
```

```
    }
}
```

```
Expected Output:
Executing Task 1 via pool-1-thread-1
Executing Task 2 via pool-1-thread-2
Executing Task 3 via pool-1-thread-1
Executing Task 4 via pool-1-thread-2
Executing Task 5 via pool-1-thread-1
```

Code Explanation:
Even though we submitted 5 tasks, only `thread-1` and `thread-2` did the work. They executed tasks 1 and 2, finished, and instantly looped back to execute 3, 4, and 5. This saves massive amounts of memory.

> ***Real-world Use:***
> *Tomcat (the software running Java web servers) uses Thread Pools. Every HTTP request sent to a website is submitted as a task to a fixed thread pool.*

Common Mistakes:

- Forgetting to call `executor.shutdown()`. If you don't shut it down, the threads will wait infinitely for new tasks, and your Java program will NEVER terminate!

Best Practices:

- Always prefer `ExecutorService` over creating raw `Thread` objects manually. It abstracts away all the dangerous low-level OS threading logic.

Practice Task:
Create a CachedThreadPool (`Executors.newCachedThreadPool()`). Submit 3 Runnable lambdas and shut it down.

11. Callable, Future, and CompletableFuture

IT STUDY HUB

Concept: Callable and Future

`Callable` is like a Runnable, but it can RETURN a result and THROW checked exceptions. `Future` is the 'receipt' you get when you submit a Callable.

> ***Real-Life Analogy:***
> *Ordering at a fast-food counter. You place an order (submit Callable). The cashier gives you a receipt with an order number (the Future). You can go sit down. When you want the food, you check the receipt (Future.get()). If the food isn't ready, you are forced to stand there and wait.*

> **Simple Explanation:**
> Because threads run in parallel, `.submit()` finishes instantly before the calculation is done. It returns a `Future` box. Later in your code, you call `.get()` on that box to extract the actual calculated result.

Syntax / Structure:

```
Callable task = () -> { return 42; };
Future future = executor.submit(task);
int result = future.get(); // BLOCKS until 42 is ready
```

Example Code:

```
import java.util.concurrent.*;

public class CallableDemo {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        // Callable: A task that returns a String
        Callable expensiveTask = () -> {
            Thread.sleep(2000); // Simulate network delay
            return "Server Data Downloaded!";
        };

        System.out.println("Submitting task...");
        Future receipt = executor.submit(expensiveTask);

        System.out.println("Task submitted. Doing other work...");
```

```
        try {
            // .get() will FREEZE the main thread here until the 2 seconds finish!
            String data = receipt.get();
            System.out.println("Result retrieved: " + data);
        } catch (Exception e) { e.printStackTrace(); }

        executor.shutdown();
    }
}
```

```
Expected Output:
Submitting task...
Task submitted. Doing other work...
(2 second pause)
Result retrieved: Server Data Downloaded!
```

Code Explanation:
The `main` thread submits the task and immediately prints 'Doing other work...'. It only freezes when it hits `receipt.get()`, blocking until the background thread finishes its sleep and returns the String.

> ***Real-world Use:***
> *Triggering 5 database queries simultaneously. You get 5 Futures back. You then wait for all 5 to complete to assemble a dashboard screen.*

Common Mistakes:

- Calling `future.get()` immediately after `.submit()`. This instantly freezes the main thread, making the execution completely sequential, entirely defeating the purpose of multithreading.

Best Practices:

- Store all your Futures in a `List`, continue doing other work, and only loop through the list to `.get()` the results at the very end of your method.

Concept: CompletableFuture (Modern Java)

An advanced Future that allows you to chain callbacks, so threads don't have to block and wait.

> ***Real-Life Analogy:***
> *Instead of checking your receipt at the counter (`Future.get()`), you give the cashier a pager. When the food is ready, the pager buzzes and handles it automatically without you ever standing at the counter.*

> **Simple Explanation:**
> `CompletableFuture` uses functional programming (Lambdas) to say: 'Run Task A. THEN when it's done, automatically pass the result to Task B.'

Syntax / Structure:

```
CompletableFuture.supplyAsync(() -> "Data")
                 .thenAccept(result -> System.out.println(result));
```

Code Explanation:
We use `.supplyAsync()` to run a background task. `.thenAccept()` is a Consumer that automatically triggers the exact millisecond the background task finishes. The main thread never freezes!

> ***Real-world Use:***
> *Modern high-performance web applications (Spring WebFlux) use CompletableFutures extensively to ensure the server never freezes while waiting for database queries to finish.*

> **Common Mistakes:**
>
> -

> **Best Practices:**
>
> - In modern Java 8+ development, always prefer `CompletableFuture` over standard `Future` when building complex asynchronous pipelines.

12. Concurrent Collections and Atomic Variables

Standard collections like `ArrayList` and `HashMap` are **NOT thread-safe**. If two threads modify them simultaneously, they will crash (e.g., `ConcurrentModificationException`) or corrupt data. Java provides a specialized `java.util.concurrent` package to solve this.

Concurrent vs Normal Collections

Normal Collection (Unsafe)	Concurrent Collection (Safe)	How it achieves safety
`HashMap`	`ConcurrentHashMap`	Lock Striping. It locks only the specific 'bucket' being updated, not the whole map. Very fast!
`ArrayList`	`CopyOnWriteArrayList`	Every time you write/add, it creates a fresh clone of the underlying array. Great for reading, slow for writing.

IT STUDY HUB

Concept: Atomic Variables

Variables that provide lock-free thread-safe operations.

> ***Real-Life Analogy:***
> *A turnstile at a subway station. It mathematically guarantees that exactly one person passes through at a time, mechanically preventing two people from squishing through together, without needing a security guard (Lock) to orchestrate it.*

> **Simple Explanation:**
> Instead of wrapping `count++` in a slow `synchronized` block, you use an `AtomicInteger`. It utilizes low-level CPU architecture (Compare-And-Swap) to update values safely without locking threads.

Syntax / Structure:

```
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // Thread-safe count++
```

Example Code:

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicDemo {
    public static void main(String[] args) throws InterruptedException {
        AtomicInteger safeCount = new AtomicInteger(0);

        Runnable task = () -> {
            for(int i=0; i<1000; i++) safeCount.incrementAndGet();
        };

        Thread t1 = new Thread(task); Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Atomic Count: " + safeCount.get()); // 2000!
    }
}
```

```
Expected Output:
Atomic Count: 2000
```

Code Explanation:
We achieved the exact same safety as the `synchronized` keyword, but `incrementAndGet()` is significantly faster because it operates without locking the thread.

> ***Real-world Use:***
> *High-speed metrics counters, generating unique sequence IDs, and managing flags in distributed systems.*

> **Common Mistakes:**
>
> - Using Atomics for complex logic involving multiple variables. Atomics only guarantee safety for *that single variable*. If you need to update two variables together safely, you must use a Lock.

> **Best Practices:**
>
> - If you only need to protect a simple counter or a boolean flag, always use `AtomicInteger` or `AtomicBoolean` instead of `synchronized`.

---

13. Project: Multithreaded Task-Processing Application

Let's synthesize everything. We will build an application that simulates a server receiving 5 heavy network requests. Instead of processing them sequentially (taking 10 seconds), we will use an **ExecutorService** to process them concurrently (taking 2 seconds). We will use **Callable** to return the status of the requests, and **AtomicInteger** to safely track how many succeeded.

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

// 1. The Task (Callable allows returning a String result)
class NetworkRequest implements Callable {
    private String requestName;
    private AtomicInteger successCounter; // Shared resource

    public NetworkRequest(String name, AtomicInteger counter) {
        this.requestName = name;
        this.successCounter = counter;
    }

    @Override
    public String call() {
        System.out.println(Thread.currentThread().getName() + " processing " + requestNa
        try {
            // Simulate network delay (1-2 seconds)
            int processingTime = (int) (Math.random() * 1000) + 1000;
            Thread.sleep(processingTime);

            // Randomly simulate a network failure
            if (Math.random() > 0.8) {
                throw new Exception("Connection Timeout!");
            }

            // Safely increment shared counter
            successCounter.incrementAndGet();
```

```
            return requestName + " SUCCESS in " + processingTime + "ms";

        } catch (Exception e) {
            return requestName + " FAILED: " + e.getMessage();
        }
    }
}

public class ServerApplication {
    public static void main(String[] args) {
        System.out.println("=== SERVER BOOTING UP ===");

        // 2. Setup Thread Pool & Shared Resources
        ExecutorService pool = Executors.newFixedThreadPool(3);
        AtomicInteger globalSuccessCounter = new AtomicInteger(0);

        // Box to hold all our Futures (receipts)
        List> receipts = new ArrayList<>();

        long startTime = System.currentTimeMillis();

        // 3. Submit 5 Concurrent Tasks
        System.out.println("--- Dispatching 5 Requests ---");
        for (int i = 1; i <= 5; i++) {
            NetworkRequest task = new NetworkRequest("Request-" + i, globalSuccessCounte
            Future receipt = pool.submit(task);
            receipts.add(receipt); // Store receipt for later
        }

        // 4. Retrieve Results (Blocking until complete)
        System.out.println("\n--- Aggregating Results ---");
        for (Future receipt : receipts) {
            try {
                // .get() blocks until that specific task finishes
                String result = receipt.get();
                System.out.println(result);
            } catch (Exception e) {
                System.out.println("Error extracting Future data.");
            }
        }

        // 5. System Shutdown & Metrics
        pool.shutdown();
```

```
        long endTime = System.currentTimeMillis();

        System.out.println("\n=== SERVER SHUTDOWN ===");
        System.out.println("Total Successful Requests: " + globalSuccessCounter.get() +
        System.out.println("Total Execution Time: " + (endTime - startTime) + "ms");
        // Sequentially, this would take ~7500ms. With 3 threads, it takes ~3000ms!
    }
}
```

```
=== SERVER BOOTING UP ===
--- Dispatching 5 Requests ---
pool-1-thread-1 processing Request-1...
pool-1-thread-3 processing Request-3...
pool-1-thread-2 processing Request-2...

--- Aggregating Results ---
pool-1-thread-2 processing Request-4...
pool-1-thread-1 processing Request-5...
Request-1 SUCCESS in 1205ms
Request-2 FAILED: Connection Timeout!
Request-3 SUCCESS in 1840ms
Request-4 SUCCESS in 1150ms
Request-5 SUCCESS in 1900ms

=== SERVER SHUTDOWN ===
Total Successful Requests: 4/5
Total Execution Time: 3115ms
```

Project Architecture Explanation:

1. **Callable:** Instead of `Runnable`, we used `Callable`. This allowed our network task to return a definitive success/fail String report.
2. **AtomicInteger:** We passed a shared counter to all 5 tasks. Because it is Atomic, 3 threads incrementing it simultaneously will not cause a Race Condition.
3. **Thread Pool:** We instantiated a `FixedThreadPool(3)`. Tasks 1, 2, and 3 started immediately. Tasks 4 and 5 waited in the queue until threads freed up.
4. **Future Management:** We submitted all tasks instantly and saved the `Future` receipts into a List. Then, we used a `for` loop to call `.get()`. This allowed all tasks to run in parallel in the background while we collected the results.

IT STUDY HUB

14. Module Assessment and Interview Prep

Test your mastery of Concurrency, Synchronization, and Thread Pools to prepare for senior-level technical interviews.

A. Practice Questions (Test Your Logic)

1. Write a class that implements `Runnable`, printing numbers 1 to 5. Start two threads using this runnable.
2. Explain why `Thread.sleep(1000)` must be wrapped in a try-catch block.
3. What happens if you accidentally call `t.run()` instead of `t.start()` on a thread object?
4. Write a program where the main thread prints "Start", creates a worker thread that sleeps for 2 seconds, calls `worker.join()`, and then prints "End".
5. Create a shared variable `int bankBalance = 1000;`. Write two threads that simultaneously withdraw 100 without synchronization to observe race conditions.
6. Fix the code in question 5 using the `synchronized` keyword on the withdraw method.
7. Why is a `ReentrantLock` considered more flexible than a `synchronized` method?
8. Write a program using `Executors.newFixedThreadPool(2)`. Submit 4 tasks that print the name of the executing thread.
9. Write a `Callable<Integer>` that returns the sum of 10 + 20. Submit it to an Executor and print the result using `Future.get()`.
10. What happens to the main thread when it calls `future.get()` and the background task is still running?
11. Replace a standard `int counter;` with an `AtomicInteger` and increment it inside a thread.
12. What exception is thrown if you try to call `wait()` outside of a synchronized block?
13. Identify the error: `ExecutorService pool = Executors.newFixedThreadPool(1); pool.submit(task); // Program never exits`
14. Why should you prefer `ConcurrentHashMap` over a standard `HashMap` in multithreaded environments?
15. Explain the concept of a "Deadlock" using an analogy of two locks and two keys.

IT STUDY HUB

B. University Viva Questions & Answers

Q: What is multithreading in Java?

Ans: Multithreading is a feature that allows concurrent execution of two or more parts of a program (threads) to maximize utilization of the CPU.

Q: What is the difference between a Process and a Thread?

Ans: A process is a heavyweight execution environment with its own independent memory space. A thread is a lightweight sub-process that shares the same memory space (Heap) as other threads within the same process.

Q: What are the two ways to create a thread in Java?

Ans: By extending the Thread class, or by implementing the Runnable interface (which is the preferred, more flexible approach).

Q: Why is implementing Runnable preferred over extending Thread?

Ans: Because Java only supports single class inheritance. If you extend Thread, your class cannot extend any other class. Implementing Runnable keeps your class open for other inheritance architectures.

Q: What is a Race Condition?

Ans: A concurrency flaw that occurs when two or more threads access shared data simultaneously and try to modify it, leading to unpredictable and corrupted outcomes.

Q: How does the 'synchronized' keyword work?

Ans: It places an intrinsic lock (monitor) on an object or method. Only one thread can hold the lock at a time. If Thread A holds the lock, Thread B must wait until A releases it before entering the synchronized block.

Q: What is a Deadlock?

Ans: A situation where Thread A locks Resource 1 and waits for Resource 2, while Thread B locks Resource 2 and waits for Resource 1. Both threads wait infinitely for each other, freezing the program.

Q: What is the Thread Lifecycle?

Ans: New (created but not started), Runnable (ready to run, waiting for CPU), Running (currently executing), Blocked/Waiting (waiting for a lock or a timer), and Terminated (execution complete).

IT STUDY HUB

Q: What is the difference between sleep() and wait()?

Ans: sleep() is a static method of Thread that pauses execution but KEEPS the locks it holds. wait() is an Object method used for inter-thread communication that RELEASES the lock while the thread goes to sleep.

Q: What is the ExecutorService?

Ans: A higher-level framework from java.util.concurrent that handles thread creation, management, and task queuing using Thread Pools, abstracting away raw Thread instantiation.

Q: Why use a Thread Pool instead of creating new threads for every task?

Ans: Creating threads consumes significant OS resources and time. A thread pool reuses a fixed number of threads, preventing memory exhaustion (OutOfMemoryError) during high-traffic scenarios.

Q: What is the difference between Runnable and Callable?

Ans: Runnable has a run() method that returns void and cannot throw checked exceptions. Callable has a call() method that can return a value (wrapped in a Future) and can throw checked exceptions.

Q: What is a Future object?

Ans: An object that acts as a placeholder for a result that will be generated asynchronously in the future. You use future.get() to extract the result when it's ready.

Q: What happens if you call future.get() before the task is complete?

Ans: The thread that called future.get() will block (freeze) and wait indefinitely until the background task finishes and provides the result.

Q: What is an Atomic Variable (e.g., AtomicInteger)?

Ans: Variables from java.util.concurrent.atomic that use low-level CPU Compare-And-Swap (CAS) instructions to safely update values concurrently without the performance overhead of using 'synchronized' locks.

Q: What is a ConcurrentHashMap?

Ans: A highly optimized, thread-safe implementation of HashMap. It uses lock-striping (locking only portions of the map rather than the entire object) to allow multiple threads to read and write safely and quickly.

IT STUDY HUB

Q: What is the difference between notify() and notifyAll()?

Ans: notify() wakes up a single random thread that is waiting on an object's monitor. notifyAll() wakes up ALL threads waiting on that monitor, letting them compete for the lock.

Q: What is a ReentrantLock?

Ans: An explicit lock implementation that offers more advanced features than 'synchronized', such as the ability to back out if a lock isn't available (tryLock), fairness policies, and explicit unlock() calling.

Q: Why MUST ReentrantLock.unlock() be placed in a finally block?

Ans: To guarantee that the lock is released even if an exception occurs inside the critical section. Failing to unlock leads to permanent deadlocks.

Q: What is a Daemon Thread?

Ans: A low-priority background thread (like the Garbage Collector). The JVM will terminate automatically when all user threads finish, completely ignoring and killing any active Daemon threads.

C. Intermediate Interview Questions

Interview Q: Explain the internal mechanics of a Race Condition during a count++ operation.

Ans: The count++ operation is not atomic. It involves three machine-level steps: 1) Read the current memory value, 2) Increment the value in CPU registers, 3) Write the new value back to memory. A race condition occurs when Thread B reads the value between Thread A's step 1 and step 3. Both threads then write the same incremented value, essentially losing one count.

Interview Q: How does the 'volatile' keyword differ from 'synchronized'?

Ans: The 'volatile' keyword ensures that a variable is always read from and written to main memory, bypassing thread-local CPU caches. It guarantees visibility across threads. However, it does NOT provide mutual exclusion (atomicity). It cannot prevent race conditions on compound operations like count++. It is best used for simple boolean state flags.

IT STUDY HUB

Interview Q: What is the difference between submit() and execute() in ExecutorService?

Ans: execute() takes a Runnable and returns void. If the task throws an unhandled exception, the thread dies. submit() can take a Runnable or Callable, and it returns a Future. If the task throws an exception, it is caught and stored inside the Future, and is re-thrown when you call future.get().

Interview Q: Explain Thread Starvation.

Ans: Starvation occurs when a low-priority thread is perpetually denied CPU time because higher-priority threads are constantly monopolizing the resources. Unlike deadlock (where threads are frozen), starved threads are ready to run but are consistently ignored by the thread scheduler.

Interview Q: How do you prevent Deadlocks in your code?

Ans: 1) Lock Ordering: Ensure all threads acquire multiple locks in the exact same predefined order. 2) Lock Timeout: Use ReentrantLock.tryLock(timeout) so threads back off if they can't get all required locks. 3) Minimize Locking: Keep synchronized blocks as small as possible and avoid holding locks during long I/O operations.

Interview Q: What is a CopyOnWriteArrayList and when should you use it?

Ans: It is a thread-safe List where all mutative operations (add, set, remove) are implemented by creating a fresh copy of the underlying array. It is highly inefficient for writing, but extremely fast and thread-safe for reading. Use it in scenarios where reads vastly outnumber writes (like iterating through a list of Event Listeners).

Interview Q: Explain the concept of 'Spurious Wakeups'.

Ans: A phenomenon where a thread waiting on an object via wait() miraculously wakes up without any other thread calling notify(). Because of this OS-level quirk, wait() must ALWAYS be placed inside a while-loop condition, never a standard if-statement, so the thread re-evaluates the condition and goes back to sleep if it was a false alarm.

Interview Q: What is a CountDownLatch?

Ans: A synchronization aid from java.util.concurrent that allows one or more threads to wait until a set of operations being performed in other threads completes. It is initialized with a count. Threads call countDown(), and the main thread calls await() which blocks until the count reaches zero.

IT STUDY HUB

Interview Q: What makes CompletableFuture superior to standard Future?

Ans: A standard Future is blocking; you must call .get() and freeze the thread to get the result. CompletableFuture is non-blocking and reactive. You can chain callbacks using .thenApply() or .thenAccept(), allowing the pipeline to execute asynchronously without ever freezing the main thread.

Interview Q: Why is String considered Thread-Safe in Java?

Ans: Because String is fully Immutable. Since its internal state cannot be changed after creation, it is mathematically impossible for multiple threads to corrupt its data via a race condition. Immutable objects are inherently thread-safe by default.

IT STUDY HUB

15. Module Summary and Key Terms

Key Terms Glossary

- **Thread:** A lightweight sub-process; the smallest unit of processing executed concurrently.
- **Race Condition:** A bug where concurrent threads manipulate shared data leading to corrupted memory.
- **Synchronization:** Enforcing mutual exclusion, allowing only one thread to access a resource at a time.
- **Deadlock:** A fatal freeze where two threads indefinitely wait for locks held by each other.
- **Thread Pool:** A managed group of reusable worker threads (e.g., `ExecutorService`).
- **Atomic Operation:** An operation that runs completely independently of any other processes, guaranteeing it cannot be interrupted.

Module Summary

In this module, you broke the boundaries of sequential programming and mastered **Multithreading and Concurrency**. You learned how to spawn multiple parallel workers using `Thread` and `Runnable`, managing their lifecycles with `sleep()` and `join()`.

We faced the dangers of concurrency head-on, witnessing how unprotected shared memory leads to **Race Conditions**. You solved these bugs by applying intrinsic locks using the `synchronized` keyword and explicit locks using `ReentrantLock` and `Atomic` variables. We explored inter-thread communication via `wait/notify`.

Finally, we upgraded to enterprise-grade concurrency, using the **ExecutorService** to manage Thread Pools safely, and leveraging `Callable` and `Future` to retrieve asynchronous results in the Multithreaded Task-Processing application.

WHAT COMES NEXT?

MODULE 10 — DATABASE, JDBC & BACKEND DEVELOPMENT

Saving data to text files is great for simple scripts, but enterprise applications handle millions of records that must be queried instantly. In the next volume, we bridge the gap between Java and Relational Databases (SQL). You will master **JDBC (Java Database Connectivity)**, learning how to connect to databases, execute SQL queries, prevent SQL Injection using PreparedStatements, and build a robust backend architecture.

IT STUDY HUB SERIES